UNIVERSITY OF WOLLONGONG IN DUBAI

Assignment Cover Sheet

Subject Code:	**CSCI316**
Subject Name:	Big Data Mining Techniques and Implementation
Submission Type:	Project Report
Assignment Title:	**CSCI 316 Project – 2 (Transfer Learning)**
Student Name:	Saadiyah Asif, Sadhguna Kumar, Zainab Gawai, Nahan Salam, Noura Morsy, Frenica, Rahma, Laila, Ziya Zanober, Guzail Rafi
Student Number:	8416655 | 8448899 | 88540238 | 8226337 | 8877725 | 8488885 | 8540184 | 8347050 | 8417337 | 7926972
Student Email:	sa841@uowmail.edu.au, svsk777@uowmail.edu.au, zgz045@uowmail.edu.au, ns732@uowmail.edu.au, Gr515@uowmail.edu.au, Lme131@uowmail.edu.au, raa130@uowmail.edu.au, fjd878@uowmail.edu.au, Zz162@uowmail.edu.au
Lecturer Name:	Dr. Patrick
Due Date:	20/03/2026
Date Submitted:	20/03/2026

PLAGIARISM:
The penalty for deliberate plagiarism is FAILURE in the subject. Plagiarism is cheating by using the written ideas or submitted work of someone else. UOWD has a strong policy against plagiarism.
The University of Wollongong in Dubai also endorses a policy of non-discriminatory language practice and presentation.

PLEASE NOTE: STUDENTS MUST RETAIN A COPY OF ANY WORK SUBMITTED

DECLARATION:
I/We certify that this is entirely my/our own work, except where I/we have given fully-documented references to the work of others, and that the material contained in this document has not previously been submitted for assessment in any formal course of study. I/we understand the definition and consequences of plagiarism.

Signature of Student: Saadiyah Asif, Sadhguna Kumar, Zainab Gawai, Nahan Salam, Noura Morsy, Frenica, Rahma, Laila, Ziya Zanober, Guzail Rafi

Optional Marks:

Comments:

Lecturer Assignment Receipt (To be filled in by student and retained by Lecturer upon return of assignment)

Subject: **Assignment Title:**
Student Name: **Student Number:**
Due Date: **Date Submitted:**
Signature of Student:

Student Assignment Receipt (To be filled in and retained by Student upon submission of assignment)

Subject: CSCI316 Big Data Mining Techniques and Implementation
Assignment Title: CSCI 316 Project – 2
Student Name: Saadiyah Asif, Sadhguna Kumar, Zainab Gawai, Nahan Salam, Noura Morsy, Frenica, Rahma, Laila, Ziya Zanober, Guzail Rafi
Student Number: 8416655 | 8448899 | 88540238 | 8226337 | 8877725 | 8488885 | 8540184 | 8347050 | 8417337 | 7926972
Due Date: 20/03/2026
Date Submitted: 20/03/2026
Signature of Lecturer

Table of Contents

Introduction 2
Problem Statement 2
Task and Language Justification 2
Implementation (Arabic – English) 3
- Data Description 3
- Methodology 4
  - Full Fine-Tuning 4
  - PEFT/Lora 8
  - Experiments and Results 13
  - Framework (PyTorch + Transformers) 16

Implementation (Bangla-Hindi-English) 17
Data Description 17
Methodology 21
- Full Fine-Tuning (Trainer API) 21
- Full Fine-Tuning (Native PyTorch) 23
- Parameter-Efficient Fine-Tuning (PEFT) (Trainer API) 24
- Parameter-Efficient Fine-Tuning (PEFT) (Native PyTorch) 26

Experiments and Results 28
- Full Fine-Tuning Results using Classification Report 28
- Full Fine-Tuning Results (Native PyTorch) using Classification Report 30
- PEFT Results (Trainer API) using Classification Report 32
- PEFT Results (Native PyTorch) using Classification Report 33

Cultural Evaluation 35
Discussion 35
Conclusion and Future Work 36
References 38
Appendix 38

Introduction

This project implements transfer learning for low-resource languages using a pre-trained Large Language Model (LLM) called XLM-Roberta-Base. The aim of this project is to tackle the linguistic inequality in the field of AI, especially Natural Language Processing (NLP) for certain languages. In this project, we test our transfer learning approach for the Hindi and Arabic languages.

Problem Statement

This project focuses on how we can adapt pre-trained multilingual LLMs to effectively handle sentiment analysis in low-resource code-switched text, specifically targeting intra-sentential language switching. The report is divided into two sections where the base model is fine tuned for different low-resource language tasks: Arabic sentiment analysis and Hindi-Bangla-English code-switched sentiment analysis.

Task and Language Justification

1. **Why is transfer learning necessary for your chosen task?**
   Reliable sentiment datasets are scarce. Most publicly available ones are either too small (30-40 rows), synthetically generated without human review, or incorrectly labelled. Training from scratch would require a massive sentiment-labelled and verified corpus which is not possible for low-resource languages. Additionally, quality code-mixed datasets were found for NER and Language Identification for both Arabic and Hindi but not for sentiment analysis.

2. **What specific knowledge is being transferred and where does it reside in the pre-trained model?**
   XLM-RoBERTa is pre-trained on 2.5TB of data across 100 languages, encoding rich multilingual representations including sub-word semantics, grammar, and cross-lingual relationships. The knowledge transferred is these multilingual representations which are already embedded in the model's weights. Fine-tuning adapts these representations to sentiment classification by updating the model weights on the task-specific labelled data.

   For Hindi, this means the model's existing understanding of Devanagari script, Bengali, and English sub-word patterns is redirected toward identifying sentiment in code-mixed sentences. For Arabic, the model's pre-trained Arabic script representations are adapted to handle dialectal Egyptian Arabic mixed with English which would not be possible to learn from the 4,100 samples alone from scratch.

3. **What are the potential failure modes when transferring to your target language/dialect?**
   For both languages, heavily fragmented rare words may lose semantic meaning when split into many sub-words by the model's SentencePiece Tokenizer, although hard OOV is avoided. Sarcasm and indirect sentiment expression are particularly challenging in both Arabic dialects and code-mixed Hindi text. Additionally, since XLM-RoBERTa was pre-trained on more data for

some languages than others, it may rely more heavily on English patterns, limiting its true understanding of the low-resource languages.

Implementation (Arabic – English)

Fine-tuning is considered a subset of transfer-learning. It involves leveraging the existing knowledge of a pre-trained model as the starting point for learning new tasks. This can largely reduce the expensive computing power that would have been required to train deep learning models with billions of parameters, from scratch. In the below section, we implement full fine tuning as well PEFT. We also demonstrate implementation across two frameworks: HuggingFace Trainer API and Native PyTorch.

Data Description

For the Arabic component of this project, the EESA Corpus was used as the main dataset for sentiment classification. This corpus consists of Arabic-English code-switched comments annotated for sentiment and is provided in three predefined splits: training, development, and test. The training set contains 2,464 comments, while both the development and test sets contain 818 comments each.

Each sample in the corpus contains a code-switched comment and its corresponding sentiment label. The dataset uses a three-class sentiment scheme:

- negative
- neutral
- positive

The EESA Corpus is well suited to this task because it contains naturally occurring code-switched text and human-assigned sentiment labels, allowing direct use in supervised sentiment classification. This is particularly important for evaluating transfer learning in realistic multilingual settings, where sentiment must be inferred from mixed-language informal text.

Several characteristics of the dataset make the task challenging. The comments contain intra-sentential code-switching between Arabic and English, introducing lexical and structural variability. In addition, the Arabic text includes informal and dialectal usage, which may differ from standard written Arabic. The dataset is also moderately imbalanced, with the positive class being more frequent than the negative and neutral classes.

A preprocessing pipeline was applied before model training to reduce noise while preserving meaningful sentiment and code-switching patterns. The preprocessing steps included the removal of URLs and user mentions, whitespace normalisation, and light Arabic normalisation, including the standardisation of different forms of the letter Alef and the conversion of ى to ي. The sentiment labels were then encoded numerically as 0 for negative, 1 for neutral, and 2 for positive.

Following preprocessing, the data was converted into Hugging Face dataset format and tokenised using the XLM-RoBERTa tokenizer. Tokenisation was performed with truncation and padding to a maximum sequence length of 128 tokens. The processed datasets were then formatted as PyTorch tensors for downstream model training, validation, and testing.

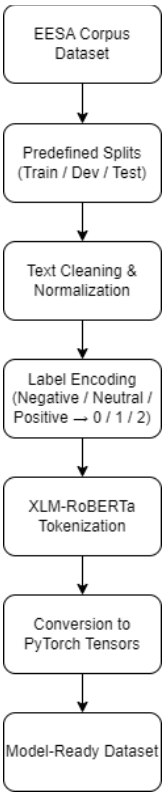


Fig 1.1. Preprocessing pipeline for EESA code-switched Arabic-English sentiment data.

Methodology

We implemented two distinct transfer learning approaches: Full Fine-Tuning and Parameter-Efficient Fine-Tuning (PEFT using LoRA) for Arabic sentiment analysis.

The experiments were conducted using the HuggingFace Trainer API and PyTorch, leveraging the multilingual capabilities of the XLM-RoBERTa model.

Full Fine-Tuning

Model	Task	Test Accuracy	F1-Score
XLM-RoBERTa-base	Arabic Sentiment (3-class)	**86.06%**	**0.8592**

1. Dataset

The dataset was preprocessed and divided into three standard subsets for training, validation, and final evaluation:

```python
from datasets import load_from_disk

train_dataset = load_from_disk("/content/train_dataset")
val_dataset   = load_from_disk("/content/val_dataset")
test_dataset  = load_from_disk("/content/test_dataset")

print("Train:", train_dataset)
print("Val:  ", val_dataset)
print("Test: ", test_dataset)
```

```
Train: Dataset({
    features: ['label', 'input_ids', 'attention_mask'],
    num_rows: 2464
})
Val:   Dataset({
    features: ['label', 'input_ids', 'attention_mask'],
    num_rows: 818
})
Test:  Dataset({
    features: ['label', 'input_ids', 'attention_mask'],
    num_rows: 818
})
```

Each data sample contains the following fields:

- input_ids: Tokenized text representation (input IDs)
- attention_mask: Binary mask identifying valid (non-padding) tokens
- label: Integer sentiment label (0, 1, or 2)

2. Data Preprocessing

Raw Arabic text was tokenized using the XLM-RoBERTa tokenizer, which applies byte-pair encoding (BPE) suitable for multilingual input. Each sentence was converted into numerical input_ids along with a corresponding attention_mask. Padding and truncation were applied to standardize sequence lengths, enabling efficient batch processing by the transformer.

3. Model Architecture

The base model used is xlm-roberta-base, a transformer encoder pre-trained via masked language modeling on 2.5 TB of filtered CommonCrawl data across 100 languages. For this task, a classification head, a linear layer projecting the [CLS] token representation to 3 output logits, was appended on top of the encoder. The full model was fine-tuned end-to-end.

4. Handling Class Imbalance

A meaningful class imbalance exists in the training data. To prevent the model from being biased toward predicting the majority class, class weights were computed using inverse frequency and applied during training:

```python
import torch
import numpy as np
from torch import nn
from transformers import Trainer

label_array = np.array(train_dataset["label"])

class_counts = np.bincount(label_array)
total = len(label_array)
class_weights = total / (len(class_counts) * class_counts)
class_weights = torch.tensor(class_weights, dtype=torch.float)

print("Class counts:", class_counts)
print("Class weights:", class_weights)

class WeightedTrainer(Trainer):
    def __init__(self, class_weights, **kwargs):
        super().__init__(**kwargs)
        self.class_weights = class_weights.to(self.args.device)

    def compute_loss(self, model, inputs, return_outputs=False, **kwargs):
        labels = inputs.pop("labels")
        outputs = model(**inputs)
        logits = outputs.logits

        loss_fn = nn.CrossEntropyLoss(weight=self.class_weights)
        loss = loss_fn(logits, labels)

        return (loss, outputs) if return_outputs else loss
```

```
Class counts: [ 594  778 1092]
Class weights: tensor([1.3827, 1.0557, 0.7521])
```

These weights were incorporated into a custom WeightedTrainer that overrides the default loss computation with a weighted cross-entropy loss, ensuring that errors on minority-class samples contribute proportionally more to the gradient updates.

5. Training Configuration

The model was fine-tuned using the following hyperparameters. Training was conducted for 5 epochs with a batch size of 8. A learning rate of 2×10^{-5} was used along with a weight decay of 0.01 to regularize the model.

Model evaluation was performed at the end of each epoch, allowing performance monitoring throughout training. The best-performing model checkpoint was selected based on the highest weighted F1-score on the validation set.

To address class imbalance, a weighted cross-entropy loss function was used during training. This ensured that minority classes were given higher importance, improving overall model performance.

6. Evaluation Metrics

Model performance was assessed using four standard classification metrics, all computed in weighted form to account for class imbalance:

- Accuracy: proportion of correctly classified samples
- Precision: ratio of true positives to predicted positives
- Recall: ratio of true positives to actual positives
- F1-Score (weighted): harmonic mean of precision and recall, weighted by class support

7. Training Results

The table below summarizes performance per epoch on the validation set. The best checkpoint (epoch 3) is highlighted.

Part 8 — Train

[19] 12m

```
trainer.train()
```

[1540/1540 12:00, Epoch 5/5]

Epoch	Training Loss	Validation Loss	Accuracy	F1	Precision	Recall
1	0.876022	0.555241	0.782396	0.782221	0.793009	0.782396
2	0.566289	0.535324	0.825183	0.827124	0.830971	0.825183
3	0.464560	0.577894	0.842298	0.841720	0.847655	0.842298
4	0.335765	0.745383	0.837408	0.836495	0.837147	0.837408
5	0.290721	0.782514	0.852078	0.851628	0.852941	0.852078

```
Writing model shards: 100%  1/1 [00:08<00:00, 8.94s/it]
Writing model shards: 100%  1/1 [00:12<00:00, 12.27s/it]
Writing model shards: 100%  1/1 [00:22<00:00, 22.13s/it]
Writing model shards: 100%  1/1 [00:24<00:00, 24.12s/it]
Writing model shards: 100%  1/1 [00:20<00:00, 20.72s/it]
There were missing keys in the checkpoint model loaded: ['roberta.embeddings.LayerNorm.weight', 'roberta.embeddings.LayerNorm.bias', 'roberta.encoder.l
There were unexpected keys in the checkpoint model loaded: ['roberta.embeddings.LayerNorm.beta', 'roberta.embeddings.LayerNorm.gamma', 'roberta.encoder
TrainOutput(global_step=1540, training_loss=0.5066713159734553, metrics={'train_runtime': 722.1391, 'train_samples_per_second': 17.06,
'train_steps_per_second': 2.133, 'total_flos': 810389326602240.0, 'train_loss': 0.5066713159734553, 'epoch': 5.0})
```

Key observation: Training loss decreases consistently across all five epochs, confirming that the model is learning. However, validation loss begins to rise after epoch 3, indicating mild overfitting. The best checkpoint was therefore saved at epoch 3, where the F1-score peaks on the validation set.

8. Test Results

Final evaluation was performed on the held out test set using the best saved checkpoint:

[20] 5s

```
test_results = trainer.evaluate(eval_dataset=test_dataset)

print("\n===== Test Set Results =====")
for key, value in test_results.items():
    print(f"  {key}: {value:.4f}" if isinstance(value, float) else f"  {key}: {value}")
```

[103/103 00:05]

```
===== Test Set Results =====
  eval_loss: 0.7290
  eval_accuracy: 0.8606
  eval_f1: 0.8592
  eval_precision: 0.8592
  eval_recall: 0.8606
  eval_runtime: 5.6874
  eval_samples_per_second: 143.8260
  eval_steps_per_second: 18.1100
  epoch: 5.0000
```

The model generalizes well to unseen data, with test accuracy closely matching validation-set performance. The near identical precision and recall values reflect balanced class-level performance, a direct benefit of the class-weighting strategy.

9. Model Saving & Reloading

After training, the fine-tuned model and tokenizer were saved to Google Drive. The saved model was subsequently reloaded and used for inference, confirming that the serialization and storage pipeline is fully functional and that the model behavior is consistent with training-time outputs.

10. Conclusion

Fine-tuning XLM-RoBERTa-base on an Arabic 3-class sentiment dataset produced strong results, achieving 86.06% test accuracy and a weighted F1-score of 0.8592. The key findings are:

- Class-weighted cross-entropy loss effectively mitigated the impact of label imbalance
- The model converged well within five epochs, with peak validation F1 at epoch 3
- Mild overfitting was observed in later epochs but did not significantly degrade test performance
- XLM-RoBERTa's multilingual pre-training transfers effectively to Arabic NLP tasks

These results demonstrate that transformer-based fine-tuning is a practical and effective approach for Arabic sentiment analysis, even with modest dataset sizes.

PEFT/Lora

For the second Arabic experiment, we utilized Parameter-Efficient Fine Tuning (PEFT), which is a machine learning technique that improves the performance and accuracy of an LLM. One of its advantages is that it decreases computational and memory costs since it trains pre-trained LLM on a subset of a dataset. As a result, this training duration decreases where it prevents overfitting from occurring. Alongside PEFT, I used Roberta since Jais required a huge amount of resources due to its complexity. The following paragraphs will highlight each step I took to build the model.

Step 1: Install Required dependencies

```
!pip install transformers datasets peft accelerate bitsandbytes evaluate scikit-learn

Requirement already satisfied: transformers in /usr/local/lib/python3.12/dist-packages (5.0.0)
Requirement already satisfied: datasets in /usr/local/lib/python3.12/dist-packages (4.0.0)
Requirement already satisfied: peft in /usr/local/lib/python3.12/dist-packages (0.18.1)
Requirement already satisfied: accelerate in /usr/local/lib/python3.12/dist-packages (1.13.0)
Collecting bitsandbytes
  Downloading bitsandbytes-0.49.2-py3-none-manylinux_2_24_x86_64.whl.metadata (10 kB)
Collecting evaluate
  Downloading evaluate-0.4.6-py3-none-any.whl.metadata (9.5 kB)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from transformers) (3.25.2)
Requirement already satisfied: huggingface-hub<2.0,>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (1.7.1)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from transformers) (2.0.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (26.0)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from transformers) (6.0.3)
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/dist-packages (from transformers) (2025.11.3)
```

The transformer library provides the pre-trained models, tokenizers, and training pipelines. Next, the dataset library allows us to load and pre-process datasets from hugging face. PEFT library is the main library out of all. It gives the individual the ability to fine tune a model by training only a section of data. The accelerate library handles the GPU run time. Bitsandbytes enables quantization to reduce GPU memory usage. The evaluate library provides the metrics for our analysis purposes. Lastly, scikit-learn is a famous library used for splitting the dataset.

Step 2: Load the Dataset

```
from datasets import load_from_disk

train_tok = load_from_disk("/content/train_dataset")
val_tok   = load_from_disk("/content/val_dataset")
test_tok  = load_from_disk("/content/test_dataset")

print("Train:", train_tok)
print("Val:  ", val_tok)
print("Test: ", test_tok)
```

```
Train: Dataset({
    features: ['label', 'input_ids', 'attention_mask'],
    num_rows: 2464
})
Val:   Dataset({
    features: ['label', 'input_ids', 'attention_mask'],
    num_rows: 818
})
Test:  Dataset({
    features: ['label', 'input_ids', 'attention_mask'],
    num_rows: 818
})
```

The above code is showing how the dataset is loaded from the huggingface library. Then, it outputs a summary of how much data is in the training, and test portions.

Step 3: Convert huggingface dataset columns into pytorch tensors

```
train_tok.set_format("torch")
val_tok.set_format("torch")
test_tok.set_format("torch")
```

The following code uses the .set_format function to convert the huggingface dataset into pytorch tensors.

Step 4: Load Model

```
from transformers import AutoModelForSequenceClassification

model = AutoModelForSequenceClassification.from_pretrained(
    "xlm-roberta-base",
    num_labels=3
)
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret in your Google Colab and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
config.json: 100% [bar] 615/615 [00:00<00:00, 67.9kB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
model.safetensors: 100% [bar] 1.12G/1.12G [00:04<00:00, 386MB/s]
Loading weights: 100% [bar] 197/197 [00:00<00:00, 625.63it/s, Materializing param=roberta.encoder.layer.11.output.dense.weight]
XLMRobertaForSequenceClassification LOAD REPORT from: xlm-roberta-base
```

This code shows how we loaded XLM-ROBERTa model for 3 class classification

Step 5: Apply LoRA

```
from peft import LoraConfig, get_peft_model, TaskType

lora_config = LoraConfig(
    r=8,
    lora_alpha=16,
    lora_dropout=0.1,
    task_type=TaskType.SEQ_CLS,
    target_modules=["query", "value"],
    modules_to_save=["classifier"]
)

model = get_peft_model(model, lora_config)
model.print_trainable_parameters()
```

```
trainable params: 887,811 || all params: 278,933,766 || trainable%: 0.3183
```

We are applying Lora configuration with ranking 8. To decrease the training time, we added a lora dropout layer with a value of 0.1.

Step 6: Define Metrics

```
import numpy as np
import pandas as pd
from sklearn.metrics import accuracy_score, precision_recall_fscore_support

epoch_logs = []

def compute_metrics(eval_pred):
    logits, labels = eval_pred
    predictions = np.argmax(logits, axis=-1)

    precision, recall, f1, _ = precision_recall_fscore_support(
        labels, predictions, average="weighted", zero_division=0
    )
    acc = accuracy_score(labels, predictions)

    return {
        "accuracy": acc,
        "f1": f1,
        "precision": precision,
        "recall": recall
    }
```

This code defines the evaluation function used after each epoch. It computes accuracy, weighted F1, precision, and recall using scikit-learn, which the Trainer calls automatically during evaluation.

Step 7: Weighttrainer Function

```python
import torch
import numpy as np
from torch import nn
from transformers import Trainer

# Compute class weights from training set
label_list = train_tok["label"] if hasattr(train_tok["label"], "__iter__") else list(train_tok["label"])
label_array = np.array(label_list)

class_counts = np.bincount(label_array)
total = len(label_array)
class_weights = total / (len(class_counts) * class_counts)
class_weights = torch.tensor(class_weights, dtype=torch.float)

print("Class counts:", class_counts)
print("Class weights:", class_weights)

# Define WeightedTrainer
class WeightedTrainer(Trainer):
    def __init__(self, class_weights, **kwargs):
        super().__init__(**kwargs)
        self.class_weights = class_weights.to(self.args.device)

    def compute_loss(self, model, inputs, return_outputs=False, **kwargs):
        labels = inputs.pop("labels")
        outputs = model(**inputs)
        logits = outputs.logits

        loss_fn = nn.CrossEntropyLoss(weight=self.class_weights)
        loss = loss_fn(logits, labels)

        return (loss, outputs) if return_outputs else loss
```

```
Class counts: [ 594  778 1092]
Class weights: tensor([1.3827, 1.0557, 0.7521])
```

The weighttrainer class calculates and readjusts the weights for each class to handle the imbalanced dataset (594 / 778 / 1092 samples across 3 classes). As a result, it made all classes have a balanced sample of 1092.

Step 8: Training configuration

```python
from transformers import TrainingArguments

training_args = TrainingArguments(
    output_dir="./results_lora",
    logging_dir="./logs_lora",

    num_train_epochs=5,
    per_device_train_batch_size=8,
    per_device_eval_batch_size=8,


    learning_rate=3e-4,
    weight_decay=0.01,


    eval_strategy="epoch",
    save_strategy="epoch",
    logging_strategy="epoch",
    logging_steps=1,

    load_best_model_at_end=True,
    metric_for_best_model="f1",

    report_to="none"
)
```

```
`logging_dir` is deprecated and will be removed in v5.2. Please
```

This code sets all training hyperparameters: 5 epochs, batch size of 8, learning rate of 3e-4, weight decay of 0.01. Evaluation and model saving happen every epoch, with the best model selected based on F1 score.

Step 9: Create Trainer

```
from transformers import Trainer

trainer = WeightedTrainer(
    class_weights=class_weights,
    model=model,
    args=training_args,
    train_dataset=train_tok,
    eval_dataset=val_tok,
    compute_metrics=compute_metrics
)
```

This code merges all the codes of WeightedTrainer with the LoRA model, training arguments, datasets, and metrics functions all together.

Step 10: Train the model

```
trainer.train()
```

[1540/1540 04:11, Epoch 5/5]

Epoch	Training Loss	Validation Loss	Accuracy	F1	Precision	Recall
1	0.920482	0.572125	0.781174	0.782916	0.787013	0.781174
2	0.631475	0.541370	0.812958	0.817090	0.834352	0.812958
3	0.543676	0.532105	0.826406	0.825557	0.827678	0.826406
4	0.481499	0.545089	0.836186	0.836076	0.836995	0.836186
5	0.426827	0.548031	0.837408	0.837452	0.838687	0.837408

```
TrainOutput(global_step=1540, training_loss=0.6007917230779475, metrics={'train_runtime':
0.6007917230779475, 'epoch': 5.0})
```

This code runs the full training loop for 5 epochs. The model improves steadily, with training loss dropping from 0.92 to 0.43. Also validation F1 improved from 0.783 to 0.837 by the final epoch.

Step 11: Evaluate on Test set

```
test_results = trainer.evaluate(eval_dataset=test_tok)

print("\n===== Test Set Results =====")
for key, value in test_results.items():
    print(f"  {key}: {value:.4f}" if isinstance(value, float) else f"  {key}: {value}")
```

```
[103/103 00:06]

===== Test Set Results =====
  eval_loss: 0.5637
  eval_accuracy: 0.8252
  eval_f1: 0.8249
  eval_precision: 0.8250
  eval_recall: 0.8252
  eval_runtime: 6.7849
  eval_samples_per_second: 120.5610
  eval_steps_per_second: 15.1810
  epoch: 5.0000
```

It gives a summary of the test results from training. The Final results were as follows

- accuracy = 82.52%
- F1 = 82.49%
- precision = 82.50%
- recall = 82.52%.

Step 12: Save model and Tokenizer

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("xlm-roberta-base")
trainer.save_model("/content/drive/MyDrive/csci316 project/lora_finetuned_xlmr")
tokenizer.save_pretrained("/content/drive/MyDrive/csci316 project/lora_finetuned_xlmr")
print("Model saved.")
```

```
tokenizer_config.json: 100%  25.0/25.0 [00:00<00:00, 616B/s]
sentencepiece.bpe.model: 100%  5.07M/5.07M [00:00<00:00, 23.5MB/s]
tokenizer.json:  9.10M/? [00:00<00:00, 18.4MB/s]
Model saved.
```

Experiments and Results

Arabic sentiment Classification

Model	Task	Test Accuracy	F1-Score
XLM-RoBERTa-base	Arabic Sentiment (3-class)	**82.5%**	**0.8249**

This section of the report highlights the discussion of our results alongside an explanation of why it occurred. In general, Parameter-efficient fine tuning is a technique where pre-trained models are trained on a specific portion of the dataset. This simplifies the training process in terms of time and computation costs.

1. Dataset

We utilized the ESSA Corpus dataset that is pre-split. The dataset was preprocessed and divided into three standard subsets for training, validation, and final evaluation:

```python
from datasets import load_from_disk

train_dataset = load_from_disk("/content/train_dataset")
val_dataset   = load_from_disk("/content/val_dataset")
test_dataset  = load_from_disk("/content/test_dataset")

print("Train:", train_dataset)
print("Val:  ", val_dataset)
print("Test: ", test_dataset)
```

```
Train: Dataset({
    features: ['label', 'input_ids', 'attention_mask'],
    num_rows: 2464
})
Val:   Dataset({
    features: ['label', 'input_ids', 'attention_mask'],
    num_rows: 818
})
Test:  Dataset({
    features: ['label', 'input_ids', 'attention_mask'],
    num_rows: 818
})
```

The dataset has a comment written in Arabic and a sentiment label attached to it. The sentiment labels are positive, negative, or neutral.

2. Data preprocessing

After the dataset was loaded, text cleaning processes were applied in terms of removing urls, normalizing whitespaces, and normalizing specific arabic variations such as Alef. Then, we used label encoding to convert sentiment string labels to numbers. At first the dataset was not balanced so we applied syntheticAugmentor function to oversample the minority classes. Lastly, we used XLM-ROBERTa tokenizer to convert the 3 classes to a proper huggingface dataset.

3. Handling Class imbalance

At the beginning, the dataset was imbalanced since the majority of sentiment labels were in favour of positive which is normal. However, this would have created a bias towards the other two sentiment labels, meaning our conclusions would not be as accurate. Therefore, we added a weighttrainer class to override the default loss computation due to allocating high weights.

```python
import torch
import numpy as np
from torch import nn
from transformers import Trainer

label_array = np.array(train_dataset["label"])

class_counts = np.bincount(label_array)
total = len(label_array)
class_weights = total / (len(class_counts) * class_counts)
class_weights = torch.tensor(class_weights, dtype=torch.float)

print("Class counts:", class_counts)
print("Class weights:", class_weights)

class WeightedTrainer(Trainer):
    def __init__(self, class_weights, **kwargs):
        super().__init__(**kwargs)
        self.class_weights = class_weights.to(self.args.device)

    def compute_loss(self, model, inputs, return_outputs=False, **kwargs):
        labels = inputs.pop("labels")
        outputs = model(**inputs)
        logits = outputs.logits

        loss_fn = nn.CrossEntropyLoss(weight=self.class_weights)
        loss = loss_fn(logits, labels)

        return (loss, outputs) if return_outputs else loss
```

```
Class counts: [ 594  778 1092]
Class weights: tensor([1.3827, 1.0557, 0.7521])
```

4. Evaluation Metrics

Model performance was assessed using four standard classification metrics, all computed in weighted form to account for class imbalance:

- Accuracy: proportion of correctly classified samples
- Precision: ratio of true positives to predicted positives
- Recall: ratio of true positives to actual positives
- F1-Score (weighted): harmonic mean of precision and recall, weighted by class support

5. Training Results

```python
trainer.train()
```

[1540/1540 04:11, Epoch 5/5]

Epoch	Training Loss	Validation Loss	Accuracy	F1	Precision	Recall
1	0.920482	0.572125	0.781174	0.782916	0.787013	0.781174
2	0.631475	0.541370	0.812958	0.817090	0.834352	0.812958
3	0.543676	0.532105	0.826406	0.825557	0.827678	0.826406
4	0.481499	0.545089	0.836186	0.836076	0.836995	0.836186
5	0.426827	0.548031	0.837408	0.837452	0.838687	0.837408

```
TrainOutput(global_step=1540, training_loss=0.6007917230779475, metrics={'train_runtim
0.6007917230779475, 'epoch': 5.0})
```

The model was trained for 5 epochs while having 3276 samples in total. It used batch size 8 and learning rate of 2e-5.

Key observations

- Training loss value decreased from 0.92 until 0.42 meaning that the model is learning well.
- Validation loss has improved from epoch 1 until 3 with a value of 0.53. However, later on for the remaining 3 epochs it increased again to 0.54 and stayed constant. This indicates that the model has started to show signs of overfitting where the model has learnt everything.
- The accuracy and F1 scores have been increasing in a good way.
- Precision and recall values are constant with values roughly ranging from 0.83. This shows that the model is balanced and not biased in any way.

6. Test Results

This screenshot displays a summary of all our obtained results from training the model.

```
test_results = trainer.evaluate(eval_dataset=test_tok)

print("\n===== Test Set Results =====")
for key, value in test_results.items():
    print(f"  {key}: {value:.4f}" if isinstance(value, float) else f"  {key}: {value}")
```

```
[103/103 00:06]

===== Test Set Results =====
  eval_loss: 0.5637
  eval_accuracy: 0.8252
  eval_f1: 0.8249
  eval_precision: 0.8250
  eval_recall: 0.8252
  eval_runtime: 6.7849
  eval_samples_per_second: 120.5610
  eval_steps_per_second: 15.1810
  epoch: 5.0000
```

7. Model Saving & Reloading

After training, the model is saved in google drives alongside all the files created from the collab files as output.

Framework (PyTorch + Transformers)

The architecture used in this work is created with the help of PyTorch and the Hugging Face Transformers library that is used to classify the text. The main idea behind this framework is to get quick processing of textual data and use a transformer-based architecture to make precise predictions.

The XLM-RoBERTa model adopted is quite appropriate to work with multilingual data as it is capable of capturing the contextual relations between the languages. This renders the framework strong and applicable to real world NLP works.

The first step is data preprocessing, which involves cleaning of raw text and converting it into a structured format. The tokenizer of XLM-RoBERTa converts text into numerical input like input ID and attention masks. Through such representations, the model is in a position to get the meaning of the sentences in addition to the structure of the sentences.

After preprocessing, the dataset is fed to PyTorch DataLoader, which splits the data into batches that are easily trained. In training, the model acquires patterns in the data and optimises its parameters. To manage the issue of class imbalance, weighted loss functions are used, and thus, the less frequent classes are learned sufficiently.

The framework examines two training strategies. The former is the full fine-tuning, whereby all the transformer model parameters are updated. The second one is parameter-efficient fine-tuning under LoRA (Low-Rank Adaptation), which only fine-tunes a few parameters. This lessens the computing demands without compromising the competitive performance.

The framework outputs give information about the framework performance. The values of the loss formed during the process of training drop with the epoch, which means that the model is training successfully. The results of the evaluation are accuracy, precision, recall, and F1-score, which all measure the predictive capability of the model. The classification report provides a precise and detailed breakdown of the performance of the various classes that the model can handle, showing how well the model handles each category.

Moreover, prediction results, i.e. correct and incorrect classifications, are visualized with the help of a confusion matrix. This is used to detect trends like wrong classification among related classes. The results indicated that the transformer-dependent structure is able to promote good classification accuracy, and LoRA provides a more efficient and competitive solution.

On the whole, the model manages to combine preprocessing, model training, and evaluation into an organized pipeline, which is scalable and suitable for text classification problems.

Implementation (Bangla-Hindi-English)

Data Description

We used a Hindi-Bangla-English code-mixed dataset which had the best sentiment labelled data for Hindi.

- Synthetic Amazon product reviews data for training and testing the model
- SentMix-3L, a naturally written code-mixed dataset which is used for evaluating the fine-tuned model. It consists of naturally written code-switched sentences, serving as the real-world generalisation benchmark.

```
Splits     : ['train', 'validation', 'test']
Features   : {'label': Value('string'), 'text': Value('string')}
Train size : 59999
Val size   : 20000
Test size  : 20000

Columns : ['label', 'text']
```

	label	text
0	Negative	DO NOT let the blu ray symbol on the box fool ...
1	Neutral	আমার কোন ভাগ্য ছিল না trying to recover my ফা...
2	Negative	यह बात वाकई में उलझ सकती है your computer. Che...

The Synthetic Amazon product reviews is the main dataset used for training. This dataset contains 99,999 code-mixed sentences which combine Hindi, English and Bangla tokens. The reviews were generated from Amazon product reviews as base text and introduced Hindi and Bangla words using code-mixing algorithms. (Nishat Raihan, 2023). The original review ratings were mapped into three sentiment classes: positive, neutral and negative.

```
#Using half the dataset to save compute
#Reduces 60k/20k/20k to 30k/10k/10k.

from sklearn.model_selection import train_test_split

df_train_raw, _ = train_test_split(df_train_raw, train_size=0.5,
                                   stratify=df_train_raw["label"], random_state=SEED)
df_val_raw,   _ = train_test_split(df_val_raw,   train_size=0.5,
                                   stratify=df_val_raw["label"],   random_state=SEED)
df_test_raw,  _ = train_test_split(df_test_raw,  train_size=0.5,
                                   stratify=df_test_raw["label"],  random_state=SEED)

print("Using half the dataset")
print(f"  Train : {len(df_train_raw):,}  {dict(df_train_raw['label'].value_counts().sort_index())}")
print(f"  Val   : {len(df_val_raw):,}    {dict(df_val_raw['label'].value_counts().sort_index())}")
print(f"  Test  : {len(df_test_raw):,}   {dict(df_test_raw['label'].value_counts().sort_index())}")
```

```
Using half the dataset
  Train : 29,999  {'Negative': np.int64(10000), 'Neutral': np.int64(10000), 'Positive': np.int64(9999)}
  Val   : 10,000    {'Negative': np.int64(3333), 'Neutral': np.int64(3333), 'Positive': np.int64(3334)}
  Test  : 10,000   {'Negative': np.int64(3333), 'Neutral': np.int64(3333), 'Positive': np.int64(3334)}
```

The dataset is pre-split into training (60k), validation (20k), and test (20k) sets, each with balanced class distribution. To reduce computational cost, 50% of each split was sampled using stratified sampling, giving a final distribution of 30k / 10k / 10k.

To evaluate the model, SentMix-3L dataset is used. This dataset contains naturally written code-switched social media comments. It has 1000 records which are labelled with the same three sentiment classes.

```
#Load natural test dataset
sentmix_hf = load_dataset("md-nishat-008/SentMix-3L")
df_sentmix  = pd.DataFrame(sentmix_hf["train"])

print(f"Shape   : {df_sentmix.shape}")
print(f"Columns : {df_sentmix.columns.tolist()}")
df_sentmix.head(3)
```

```
Shape   : (1007, 2)
Columns : ['text', 'label']
```

	text	label
0	I knew it আমি যদি work hard করি I will get pr...	Positive
1	I still can't believe that আমার সবচেয়ে কাছের ...	Positive
2	The সুন্দর sunset has made my বাজে দিন into ...	Positive

Text Processing

```python
def normalise_labels(df, label_col="label"):
    df = df.copy()
    df[label_col] = df[label_col].astype(str).str.strip().str.capitalize()
    df = df[df[label_col].isin(LABEL2ID)].dropna(subset=["text", label_col]).reset_index(drop=True)
    df["label_id"] = df[label_col].map(LABEL2ID)
    return df

df_train_raw = normalise_labels(df_train_raw)
df_val_raw   = normalise_labels(df_val_raw)
df_test_raw  = normalise_labels(df_test_raw)
df_sentmix   = normalise_labels(df_sentmix)

print("Training dataset label distribution:")
print(f"  Train : {dict(Counter(df_train_raw['label']))}")
print(f"  Val   : {dict(Counter(df_val_raw['label']))}")
print(f"  Test  : {dict(Counter(df_test_raw['label']))}")
print(f"\nSentMix-3L: {dict(Counter(df_sentmix['label']))}")
```

```
Training dataset label distribution:
  Train : {'Negative': 9999, 'Neutral': 9999, 'Positive': 9997}
  Val   : {'Neutral': 3333, 'Negative': 3332, 'Positive': 3333}
  Test  : {'Neutral': 3333, 'Negative': 3333, 'Positive': 3334}

SentMix-3L: {'Positive': 420, 'Neutral': 234, 'Negative': 353}
```

```python
class CodeSwitchedPreprocessor:
    """
    Language-aware cleaner for Bangla-Hindi-English code-mixed text.
    Preserves all three Unicode script ranges intact.
    Does NOT strip non-ASCII, transliterate, or insert <UNK>.
    """
    RE_BANGLA  = re.compile(r"[\u0980-\u09FF]+")
    RE_HINDI   = re.compile(r"[\u0900-\u097F]+")
    RE_ENGLISH = re.compile(r"[a-zA-Z]+")
    RE_URL     = re.compile(r"https?://\S+|www\.\S+")
    RE_MENTION = re.compile(r"@\w+")
    RE_HASHTAG = re.compile(r"#(\w+)")      #keep the word, drop #
    RE_REPEAT  = re.compile(r"(.)\1{3,}")   #4+ repeated chars to 2
    RE_CONTROL = re.compile(r"[\x00-\x08\x0b\x0c\x0e-\x1f\x7f]")
```

A text preprocessing pipeline was implemented to clean the dataset while ensuring that the multilingual structure of the data was maintained. The dataset has three different scripts: Latin, Devanagari and Bengali. The following text preprocessing operations were performed:

- Unicode Normalisation: Unicode Normalisation (NFC) was used to ensure that the characters are stored in a consistent format.
- Username Removed: User mentions like @username were removed during the preprocessing step.
- Removing Hashtag: The '#' symbol was removed while the original word was preserved (#happy -> happy).
- Removing Web links: Web links were removed as part of cleaning the dataset.
- Handling Repeated Characters: Repeated characters in words were reduced to two characters (soooo good -> soo good)
- Removing Control Characters: Control characters like '\n' and '\t' were removed during preprocessing.
- Removing Extra Whitespace: Extra whitespace was reduced to a single space.
- Creating Numerical Labels: The sentiment labels were mapped into numbers.

```
=== Pre-processing Demo ===

Raw    : আজকে @user visit করলাম https://t.co/xyz #HappyDay!! খুব bhala laglo!!!
Cleaned: আজকে visit করলাম HappyDay !! খুব bhala laglo!!!
Scripts: {'bangla': 0.375, 'hindi': 0.0, 'english': 0.5, 'other': 0.125}

Raw    : যহ ফিল্ম একদম #Bakwas ছিল @critic woooorst movie ever!
Cleaned: যহ ফিল্ম একদম Bakwas ছিল woorst movie ever!
Scripts: {'bangla': 0.25, 'hindi': 0.25, 'english': 0.5, 'other': 0.0}
```

Script Distribution Analysis

A simple script-detection analysis was performed to determine the proportion of tokens belonging to the three languages in the dataset.

```
print("\nScript proportions in training data (sample of 2,000):")
sample_scripts = (
    df_train_raw["clean_text"]
    .sample(2000, random_state=SEED)
    .apply(pre.detect_scripts)
    .apply(pd.Series)
)
print(sample_scripts.mean().round(3).to_string())
```

```
Script proportions in training data (sample of 2,000):
bangla     0.296
hindi      0.360
english    0.328
other      0.016
```

Hence, the dataset represents true code-switching behaviour between all 3 languages.

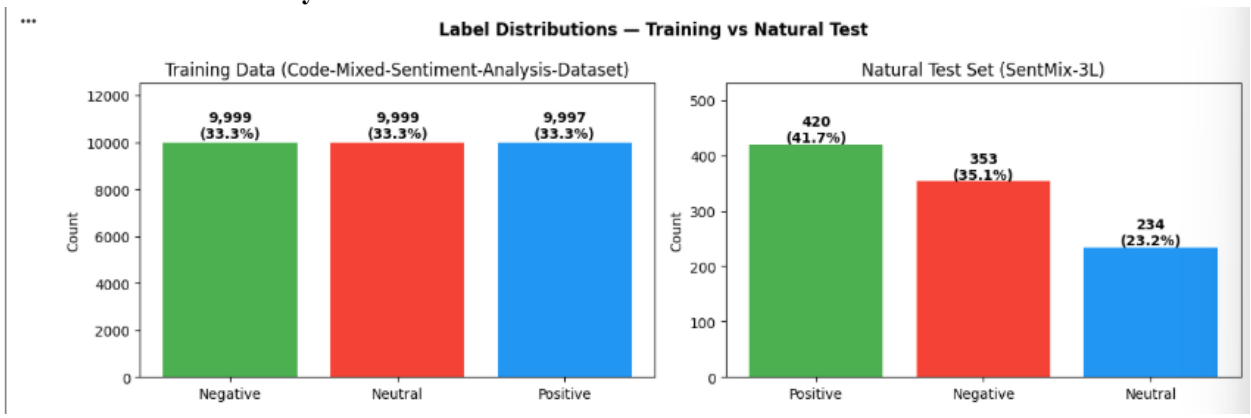
Vocabulary and OOV Analysis

An out-of-vocabulary analysis was conducted on a random sample of 5000 cleaned records to understand the lexical diversity of the dataset. Token frequency was computed to measure total unique tokens, hapax tokens and rare tokens. Hapax legomena are tokens that appear exactly once in the dataset

```
Sample size           : 5,000 texts
Total unique tokens   : 46,008
Hapax legomena (f=1)  : 27,016  (58.7%)
Rare tokens (f≤2)     : 33,160  (72.1%)
```

The number of rare tokens being large is expected in a code-switched dataset as speakers combine tokens from multiple languages and dialects. This was resolved by using SentencePiece tokenizer by XLM-Roberta, which breaks unknown words into smaller subword units.

Label Distribution Analysis



The synthetic training datasets show a balanced distribution of 33.3% of each class and around 9,999 samples.

The SentMix-3L dataset has a slightly imbalanced distribution. Positive classes have 420 samples which is 41.7% of the dataset. Negative class contains 353 samples (35.1%) and Neutral has 234 samples (23.3%). The SentMix-3L dataset has a slightly imbalanced distribution because it has naturally written code switched language.

Methodology

We implemented two-distinct transfer-learning approaches - Full Fine Tuning and PEFT (LoRA) across two frameworks: HuggingFace Trainer API and Native Pytorch for Hindi-Bangla-English data.

Full Fine-Tuning (Trainer API)

Below is the code for the training configuration.

```
CONFIG = {
    # Model
    "model_name":   "xlm-roberta-base",
    "max_length":   128,
    # Training
    "batch_size":   32,
    "num_epochs":   5,
    "learning_rate": 2e-5,
    "weight_decay":  0.01,
    "warmup_ratio":  0.1,
    # Data paths
    "train_path":        "train_cleaned.csv",
    "val_path":          "val_cleaned.csv",
    "test_synth_path":   "test_cleaned.csv",
    "test_natural_path": "sentmix_cleaned.csv",
    # Output
    "best_model_path":   "./best_full_finetuned_model",
}

LABEL2ID = {"Positive": 0, "Negative": 1, "Neutral": 2}
```

Full fine-tuning updates all 278 million parameters of XLM-RoBERTa, from the embedding layer through all 12 transformer layers to the classification head. This gives the model maximum capacity to adapt to the task but risks forgetting pre-trained multilingual knowledge - known as catastrophic forgetting, hence a lower learning rate is used.

```
def tokenize_fn(batch):
    return tokenizer(
        batch["clean_text"],
        truncation = True,
        padding    = "max_length",
        max_length = CONFIG["max_length"],
    )
```

Tokenization: SentencePiece tokenizer (250k vocabulary) handles all three scripts natively with zero OOV. Unseen words are decomposed into subword pieces rather than mapped to unknown tokens.. All sequences were padded or truncated to a maximum length of 128 tokens.

Model Architecture: A three-class classification head was added on top of the [CLS] token output, consisting of a linear layer mapping 768 dimensions down to 3 sentiment scores (Positive, Negative, Neutral). The [CLS] token represents the entire input sentence, making it the natural input to the classifier. The class with the highest score becomes the final prediction.

```python
#Training Setup

from transformers import Trainer, TrainingArguments
from sklearn.metrics import f1_score, accuracy_score

def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = np.argmax(logits, axis=-1)
    return {
        "weighted_f1": round(f1_score(labels, preds, average="weighted"), 4),
        "macro_f1":    round(f1_score(labels, preds, average="macro"),    4),
        "accuracy":    round(accuracy_score(labels, preds),               4),
    }

total_steps   = (len(train_ds) // CONFIG["batch_size"]) * CONFIG["num_epochs"]
warmup_steps  = int(CONFIG["warmup_ratio"] * total_steps)
```

```python
training_args = TrainingArguments(
    output_dir                  = "./full_ft_checkpoints",
    num_train_epochs            = CONFIG["num_epochs"],
    per_device_train_batch_size = CONFIG["batch_size"],
    per_device_eval_batch_size  = CONFIG["batch_size"],
    learning_rate               = CONFIG["learning_rate"],
    weight_decay                = CONFIG["weight_decay"],
    warmup_steps                = warmup_steps,
    eval_strategy               = "epoch",
    save_strategy               = "epoch",
    load_best_model_at_end      = True,
    metric_for_best_model       = "weighted_f1",
    greater_is_better           = True,
    logging_steps               = 200,
    fp16                        = torch.cuda.is_available(),
    seed                        = SEED,
    report_to                   = "none",
    label_names                 = ["labels"],
)
```

Training Configuration:

- **Optimizer:** AdamW with LR = 2e-5 and weight decay = 0.01. Weight decay penalises large weights during training, acting as regularisation to reduce overfitting.
- **Linear warmup** over the first 468 steps gradually increases the learning rate from near zero to 2e-5, preventing large gradient updates from destabilising the pretrained weights at the start of training.
- **fp16 mixed precision** stores some weights in 16-bit instead of 32-bit, roughly halving memory usage with no meaningful loss in accuracy.
- **Early stopping** with patience = 2 halts training if validation F1 does not improve for two consecutive epochs, restoring the best checkpoint automatically.

```
Starting full fine-tuning ...
  Training on 29,994 samples for up to 5 epochs
  Batch size : 32  |  Steps per epoch : 937
```

[4690/4690 28:55, Epoch 5/5]

Epoch	Training Loss	Validation Loss	Weighted F1	Macro F1	Accuracy
1	0.712007	0.662660	0.717700	0.717700	0.721100
2	0.616006	0.675060	0.722000	0.722000	0.720300
3	0.505728	0.655936	0.735800	0.735800	0.734500
4	0.440919	0.715219	0.728900	0.728900	0.730100
5	0.367196	0.784987	0.729500	0.729500	0.729300

Training ran for all 5 epochs, with the best checkpoint saved at **epoch 3** (val weighted F1 = 0.7358). Beyond epoch 3, validation loss began rising (from 0.655 to 0.715 to 0.784) while training loss continued decreasing (from 0.505 to 0.367), indicating overfitting. However, validation F1 remained relatively stable, suggesting the model's predictions did not degrade significantly despite the loss divergence.

Full Fine-Tuning (Native PyTorch)

This implementation uses the same base model and task setup as the Trainer API version, but with a manually implemented training loop using train_epoch() and eval_epoch() instead of letting the Trainer abstract it away. This gives full control over each training step, including the optimizer update, gradient clipping, and evaluation logic.

Model Setup: xlm-roberta-base is loaded with a 3-class sentiment head appended on top, identical to the Trainer API version. The classifier head's learning rate is intentionally set to 5x higher than the rest of the model, allowing faster adaptation of the newly added weights while keeping the pretrained layers stable. SentimentDataset class handles tokenization with a max sequence length of 128, and DataLoader pipelines are built for train, val, synthetic test, and natural test.

```python
# PyTorch Dataset and DataLoaders

class SentimentDataset(Dataset):
    def __init__(self, texts, labels, tokenizer, max_length):
        self.texts      = texts
        self.labels     = labels
        self.tokenizer  = tokenizer
        self.max_length = max_length

    def __len__(self):
        return len(self.texts)

    def __getitem__(self, idx):
        encoding = self.tokenizer(
            self.texts[idx],
            truncation=True,
            padding="max_length",
            max_length=self.max_length,
            return_tensors="pt",
        )
        return {
            "input_ids":      encoding["input_ids"].squeeze(0),
            "attention_mask": encoding["attention_mask"].squeeze(0),
            "labels":         torch.tensor(self.labels[idx], dtype=torch.long),
        }
```

```python
def make_dataset(df):
    return SentimentDataset(
        texts=df["clean_text"].tolist(),
        labels=df["label_id"].tolist(),
        tokenizer=tokenizer,
        max_length=CONFIG["max_length"],
    )


def to_torch_dataloader(df_, shuffle=False):
    return DataLoader(
        make_dataset(df_),
        batch_size=CONFIG["batch_size"],
        shuffle=shuffle,
        num_workers=2,
        pin_memory=True,
    )


print("Building PyTorch DataLoaders ...")
train_loader      = to_torch_dataloader(train_df, shuffle=True)
val_loader        = to_torch_dataloader(val_df)
test_synth_loader = to_torch_dataloader(test_synth)
test_nat_loader   = to_torch_dataloader(test_nat)

print(f"  train_loader      : {len(train_df):,} samples")
print(f"  val_loader        : {len(val_df):,} samples")
print(f"  test_synth_loader : {len(test_synth):,} samples")
print(f"  test_nat_loader   : {len(test_nat):,} samples")
```

Key Differences from Trainer API:

- Scheduler: OneCycleLR with cosine annealing instead of linear warmup, handling the learning rate over the first 10% of steps.
- Gradient clipping at 1.0 explicitly prevents exploding gradients, which in the Trainer API is handled automatically.
- torch.amp is manually enabled for mixed precision when a GPU is available.
- Bias and LayerNorm parameters are explicitly excluded from weight decay in the optimizer grouping.

All other configuration parameters remain the same as the Trainer API version.

```python
CONFIG = {
    "model_name":    "xlm-roberta-base",
    "max_length":    128,
    "batch_size":    32,
    "num_epochs":    5,
    "learning_rate": 2e-5,
    "weight_decay":  0.01,
    "warmup_pct":    0.1,
    "max_grad_norm": 1.0,
    "patience":      2,
    "train_path":        "train_cleaned.csv",
    "val_path":          "val_cleaned.csv",
    "test_synth_path":   "test_cleaned.csv",
    "test_natural_path": "sentmix_cleaned.csv",
}

LABEL2ID = {"Positive": 0, "Negative": 1, "Neutral": 2}
ID2LABEL = {v: k for k, v in LABEL2ID.items()}
NUM_LABELS = len(LABEL2ID)
```

```python
optimizer = AdamW(optimizer_grouped_params)

scheduler = OneCycleLR(
    optimizer,
    max_lr=[CONFIG["learning_rate"], CONFIG["learning_rate"], CONFIG["learning_rate"] * 5],
    total_steps=total_steps,
    pct_start=CONFIG["warmup_pct"],
    anneal_strategy="cos",
)

criterion = nn.CrossEntropyLoss()

# Mixed precision scaler
use_amp = torch.cuda.is_available()
scaler  = torch.amp.GradScaler(enabled=use_amp)

print(f"Steps per epoch : {len(train_loader):,}")
print(f"Total steps     : {total_steps:,}")
print(f"Warmup steps    : {int(total_steps * CONFIG['warmup_pct']):,}")
print(f"Trainable params: {trainable_params:,} ({pct:.1f}% of total)")
```

```
Steps per epoch : 938
Total steps     : 4,690
Warmup steps    : 469
Trainable params: 278,045,955 (100.0% of total)
```

Parameter-Efficient Fine-Tuning (PEFT) (Trainer API)

In addition to full fine-tuning, we implemented a parameter-efficient alternative using Low-Rank Adaptation (LoRA). Unlike full fine-tuning which updates all 278 million parameters, LoRA freezes the entire pretrained model and injects small trainable adapter matrices into the attention layers. For each targeted weight matrix W, the update is:

$$W' = W_frozen + (\alpha / r) * B * A$$

where A and B are small low-rank matrices. Only these are trained while everything else remains frozen, adapting the model at a fraction of the computational cost.

Model Setup: The base model was loaded using AutoModelForSequenceClassification with a three-class classification head appended on top. A LoraConfig was then applied using the PEFT library to wrap the base model with LoRA adapters.

```python
base_model = AutoModelForSequenceClassification.from_pretrained(
    "xlm-roberta-base",
    num_labels              = 3,
    id2label                = ID2LABEL,
    label2id                = LABEL2ID,
    ignore_mismatched_sizes = True,
)

lora_config = LoraConfig(
    task_type      = TaskType.SEQ_CLS,
    r              = 8,        # LoRA rank
    lora_alpha     = 16,       # scaling: alpha/r = 2x
    lora_dropout   = 0.1,
    target_modules = ["query", "key", "value", "dense"],
    bias           = "none",
)

model = get_peft_model(base_model, lora_config)
```

The LoRA adapters were applied to all four attention projection matrices (query, key, value, and dense) across all 12 transformer layers of XLM-RoBERTa. This covers the full attention mechanism rather than just query and value, giving the model more flexibility to adapt cross-lingual attention patterns to the sentiment task.

Parameter Breakdown: A key advantage of LoRA is that the vast majority of the model remains frozen.

```
────────────────────────────────────────────────────────
  LoRA Parameter Breakdown
────────────────────────────────────────────────────────
  Total parameters       :  279,965,958
  Trainable (LoRA only)  :    1,920,003  (0.686%)
  Frozen (XLM-R base)    :  278,045,955
────────────────────────────────────────────────────────
  LoRA rank (r)          : 8
  LoRA alpha             : 16
  Effective scale (α/r)  : 2.0×
  Target modules         : ['query', 'key', 'value', 'dense']
────────────────────────────────────────────────────────
```

Only 1.92M parameters (0.686%) are trained, hence LoRA requires significantly less GPU memory and trains faster than full fine-tuning. The frozen XLM-RoBERTa weights preserve the multilingual knowledge from pretraining, reducing the risk of catastrophic forgetting.

Training Configuration

Training was managed using the HuggingFace Trainer API with the following configuration:

- **Learning Rate = 1e-4** is higher than full fine-tuning since only the small adapter matrices are being updated, so a larger learning rate trains them faster without any risk of destabilising the frozen pretrained weights.
- **Linear warmup** over 5% of total steps to prevent large gradient updates at the start of training.
- **Max length = 192 tokens** is increased from 128 to accommodate longer code-switched sentences, which tokenise into more subword pieces when mixing multiple scripts.
- Mixed precision training (fp16) was enabled to reduce memory usage.

```python
CONFIG = {
    # Model
    "model_name":     "xlm-roberta-base",
    "max_length":     192,
    # Training
    "batch_size":     32,
    "num_epochs":     5,
    "learning_rate":  1e-4,
    "weight_decay":   0.01,
    "warmup_ratio":   0.05,
    # LoRA
    "lora_r":         8,
    "lora_alpha":     16,
    "lora_dropout":   0.1,
    "target_modules": ["query", "key", "value", "dense"],
    # Data paths (output of Notebook 1)
    "train_path":        "train_cleaned.csv",
    "val_path":          "val_cleaned.csv",
    "test_path":         "test_cleaned.csv",
    "test_tweets_path":  "sentmix_cleaned.csv",
}

LABEL2ID = {"Positive": 0, "Negative": 1, "Neutral": 2}
ID2LABEL  = {v: k for k, v in LABEL2ID.items()}
```

```python
training_args = TrainingArguments(
    output_dir                  = "./lora_checkpoints",
    num_train_epochs            = CONFIG["num_epochs"],
    per_device_train_batch_size = CONFIG["batch_size"],
    per_device_eval_batch_size  = CONFIG["batch_size"],
    learning_rate               = CONFIG["learning_rate"],
    weight_decay                = CONFIG["weight_decay"],
    warmup_ratio                = CONFIG["warmup_ratio"],
    eval_strategy               = "epoch",
    save_strategy               = "epoch",
    load_best_model_at_end      = True,
    metric_for_best_model       = "weighted_f1",
    greater_is_better           = True,
    logging_steps               = 100,
    fp16                        = torch.cuda.is_available(),
    seed                        = SEED,
    report_to                   = "none",
    label_names                 = ["labels"],
)

trainer = Trainer(
    model           = model,
    args            = training_args,
    train_dataset   = train_ds,
    eval_dataset    = val_ds,
    compute_metrics = compute_metrics,
)
```

The best checkpoint was selected by validation weighted F1.

```
Training on 29,994 samples for 5 epochs
Batch size : 32  |  Steps per epoch : 937
```

[4690/4690 30:43, Epoch 5/5]

Epoch	Training Loss	Validation Loss	Weighted F1	Macro F1	Accuracy
1	0.687539	0.643764	0.719900	0.719900	0.721800
2	0.645036	0.631195	0.736400	0.736400	0.734100
3	0.597678	0.614796	0.739100	0.739100	0.738800
4	0.611797	0.625469	0.740000	0.740000	0.740900
5	0.570658	0.620238	0.743000	0.743100	0.742500

```
Training complete in 1845.6s (30.8 min)
```

Parameter-Efficient Fine-Tuning (PEFT) (Native PyTorch)

This implementation uses the same LoRA formulation as above but without the PEFT library. Using Native PyTorch requires explicit implementation of the optimizer, learning rate scheduler, gradient clipping, early stopping, and evaluation loop, which are abstracted automatically by the Trainer API.

LoRA from Scratch: Rather than calling get_peft_model(), a custom LoRALayer class wraps each targeted linear layer directly. This gives full visibility into exactly which parameters are trained and which are frozen. Adapters were injected into the query and value projections only across all 12 attention layers, unlike the Trainer API version which covered all four attention matrices.

```python
class LoRALayer(nn.Module):
    """
    Wraps a frozen Linear layer and injects trainable LoRA matrices A and B.
    Forward pass: original_output + (alpha/r) * x @ A.T @ B.T
    """
    def __init__(self, original_layer, rank=8, alpha=16, dropout=0.1):
        super().__init__()
        self.original_layer = original_layer
        self.scaling        = alpha / rank
        self.dropout        = nn.Dropout(dropout)

        in_features  = original_layer.weight.shape[1]
        out_features = original_layer.weight.shape[0]

        # Freeze original weights — they never get gradient updates
        for param in self.original_layer.parameters():
            param.requires_grad = False

        # LoRA matrices — these are the ONLY things we train
        # A initialized with small random noise, B initialized to zero
        # (so LoRA output starts at zero = same as original model)
        self.lora_A = nn.Parameter(
            torch.randn(rank, in_features) * 0.01
        )
        self.lora_B = nn.Parameter(
            torch.zeros(out_features, rank)
        )

    def forward(self, x):
        original_out = self.original_layer(x)
        lora_out     = self.dropout(x) @ self.lora_A.T @ self.lora_B.T
        return original_out + lora_out * self.scaling


def inject_lora(model, rank=8, alpha=16, dropout=0.1):
    """
    Replace query and value projections in every attention layer with LoRALayer.
    Key and dense layers left frozen — only Q and V are adapted.
    """
    injected = 0
    for layer in model.roberta.encoder.layer:
        attn = layer.attention.self
        attn.query = LoRALayer(attn.query, rank, alpha, dropout)
        attn.value = LoRALayer(attn.value, rank, alpha, dropout)
        injected += 1
    print(f"LoRA injected into {injected} attention layers (query + value each)")
    return model
```

LoRA Parameter Breakdown: Only 887k parameters (0.319%) are trained in this implementation.

```
LoRA Parameter Breakdown (Native PyTorch)
──────────────────────────────────────────
Total parameters        :  278,340,867
Trainable (LoRA only)   :      887,811  (0.319%)
Frozen (XLM-R base)     :  277,453,056
──────────────────────────────────────────
LoRA rank (r)           : 8
LoRA alpha              : 16
Effective scale (α/r)   : 2.0×
──────────────────────────────────────────
```

Training Configuration: We give parameters similar to the exercise with Trainer API. The learning_rate is 1e-4. The linear warmup is over 5% of total training steps. The maximum sequence length was set to 102 tokens.

```python
# training setup

total_steps  = len(train_loader) * CONFIG["num_epochs"]
warmup_steps = int(total_steps * CONFIG["warmup_ratio"])

# Only pass trainable params to optimizer — frozen params excluded
optimizer = AdamW(
    [p for p in model.parameters() if p.requires_grad],
    lr=CONFIG["learning_rate"],
    weight_decay=CONFIG["weight_decay"]
)

scheduler = get_linear_schedule_with_warmup(
    optimizer,
    num_warmup_steps=warmup_steps,
    num_training_steps=total_steps
)

criterion = nn.CrossEntropyLoss()

print(f"Steps per epoch : {len(train_loader):,}")
print(f"Total steps     : {total_steps:,}")
print(f"Warmup steps    : {warmup_steps:,}")
print(f"Trainable params: {trainable_params:,} ({pct:.3f}% of total)")
```

```
Steps per epoch : 938
Total steps     : 4,690
Warmup steps    : 234
Trainable params: 887,811 (0.319% of total)
```

```
Training on 29,994 samples for up to 5 epochs
Batch size : 32  |  Steps/epoch : 938

Epoch 1/5 | Train Loss: 0.8471 | Val Loss: 0.6741 | Val F1: 0.7006 | Val Acc: 0.7032
Epoch 2/5 | Train Loss: 0.6961 | Val Loss: 0.6457 | Val F1: 0.7229 | Val Acc: 0.7213
Epoch 3/5 | Train Loss: 0.6662 | Val Loss: 0.6466 | Val F1: 0.7207 | Val Acc: 0.7228
Epoch 4/5 | Train Loss: 0.6484 | Val Loss: 0.6482 | Val F1: 0.7247 | Val Acc: 0.7267
Epoch 5/5 | Train Loss: 0.6350 | Val Loss: 0.6381 | Val F1: 0.7289 | Val Acc: 0.7301

Training complete in 4338.5s (72.3 min)
```

Experiments and Results

All variants were evaluated on two sets: the synthetically code-mixed product reviews secondary data and primary natural social media comments.

Full Fine-Tuning Results using Classification Report

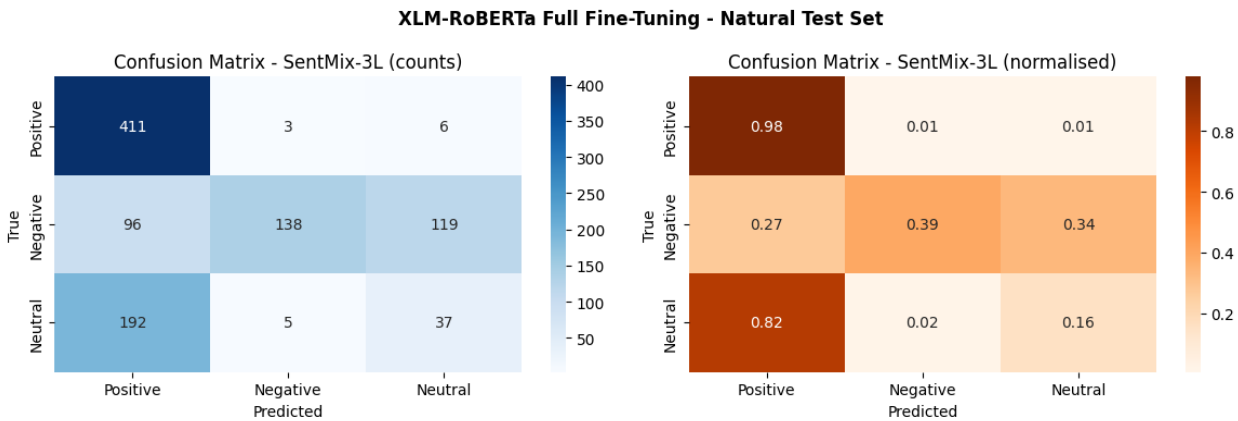
Secondary Test Set (Synthetic)

```
Evaluating on secondary test set ...

Secondary Test Set
              precision    recall  f1-score   support

    Positive     0.8133    0.8206    0.8170      3334
    Negative     0.7829    0.7021    0.7403      3333
     Neutral     0.6216    0.6802    0.6496      3333

    accuracy                         0.7343     10000
   macro avg     0.7393    0.7343    0.7356     10000
weighted avg     0.7393    0.7343    0.7356     10000
```



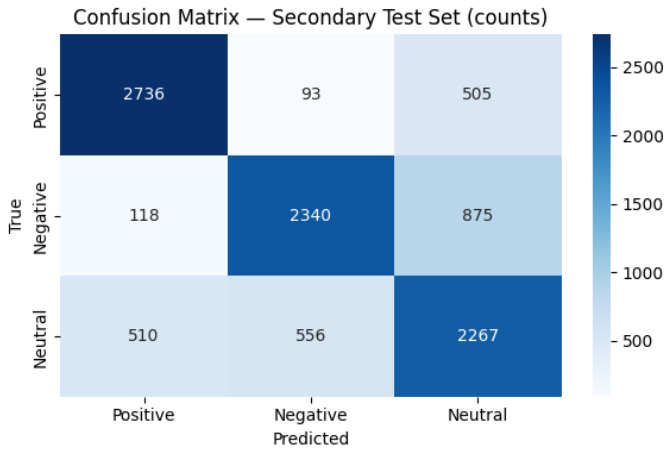
Secondary Test Set (synthetic): The model achieved a weighted F1 of 0.7356, with strongest performance on Positive (F1 = 0.817) and weakest on Neutral (F1 = 0.650), which is expected as Neutral is the most ambiguous class.

Primary Test Set (Natural)

```
SentMix-3L Natural Test Set (PRIMARY)
              precision    recall  f1-score   support

    Positive     0.6256    0.9786    0.7632       420
    Negative     0.9186    0.4476    0.6019       353
     Neutral     0.3202    0.2436    0.2767       234

    accuracy                         0.6216      1007
   macro avg     0.6215    0.5566    0.5473      1007
weighted avg     0.6573    0.6216    0.5936      1007
```



Primary Test Set (SentMix-3L - natural): On the naturally written code-switched text, weighted F1 dropped to 0.5437 — a 14-point gap reflecting domain shift between synthetic Amazon reviews and authentic code-switched text. The model heavily over-predicts Positive: recall of 0.98 on Positive but only 0.39 on Negative and 0.16 on Neutral. This suggests the model learned surface-level sentiment patterns from synthetic product reviews that do not generalise well to informal, naturally code-switched sentences where sentiment cues are more subtle and context-dependent.

Full Fine-Tuning Results (Native PyTorch) using Classification Report

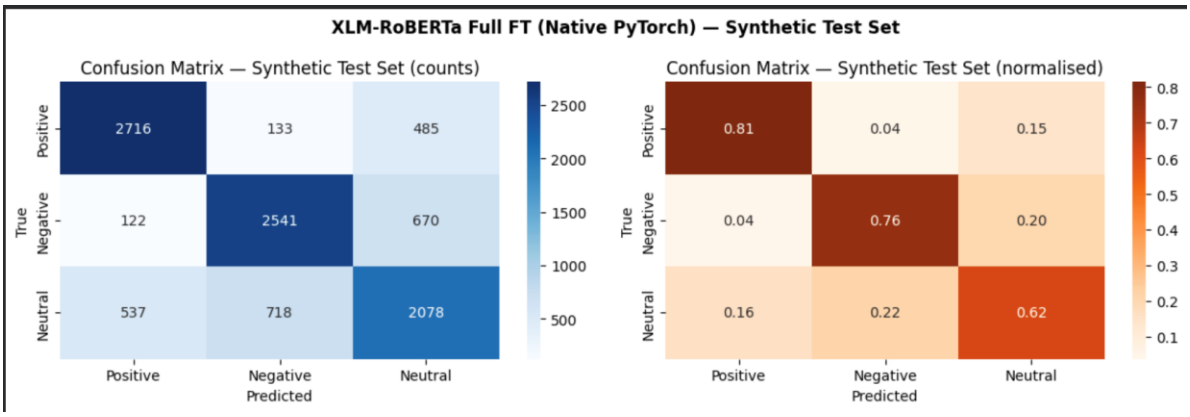
The same evaluation was performed on the Full fine tuning model that was implemented from scratch using native PyTorch training loop.

Secondary Test Set (Synthetic)

```
── Synthetic Test Set ──────────────────────────
              precision    recall  f1-score   support

    Positive     0.8047    0.8146    0.8097      3334
    Negative     0.7491    0.7624    0.7557      3333
     Neutral     0.6427    0.6235    0.6330      3333

    accuracy                         0.7335     10000
   macro avg     0.7322    0.7335    0.7328     10000
weighted avg     0.7322    0.7335    0.7328     10000
```



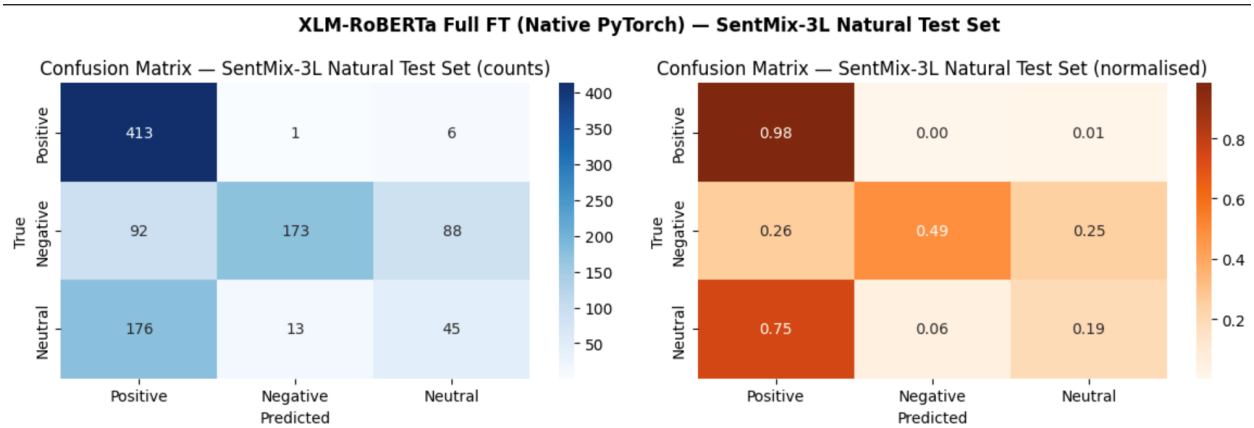
The weighted F1 in this case is 0.7328 which is again almost the same as the training data. Similar to the previous experiment, the performance was strongest on Positive (F1 = 0.8079) and weakest on Neutral (F1 = 0.6330).

Primary Test Set (SentMix-3L Natural)

```
── SentMix-3L Natural Test Set ──────────────────────────
              precision    recall  f1-score   support

    Positive     0.6065    0.9833    0.7502       420
    Negative     0.9251    0.4901    0.6407       353
     Neutral     0.3237    0.1923    0.2413       234

    accuracy                         0.6266      1007
   macro avg     0.6184    0.5552    0.5441      1007
weighted avg     0.6525    0.6266    0.5936      1007
```



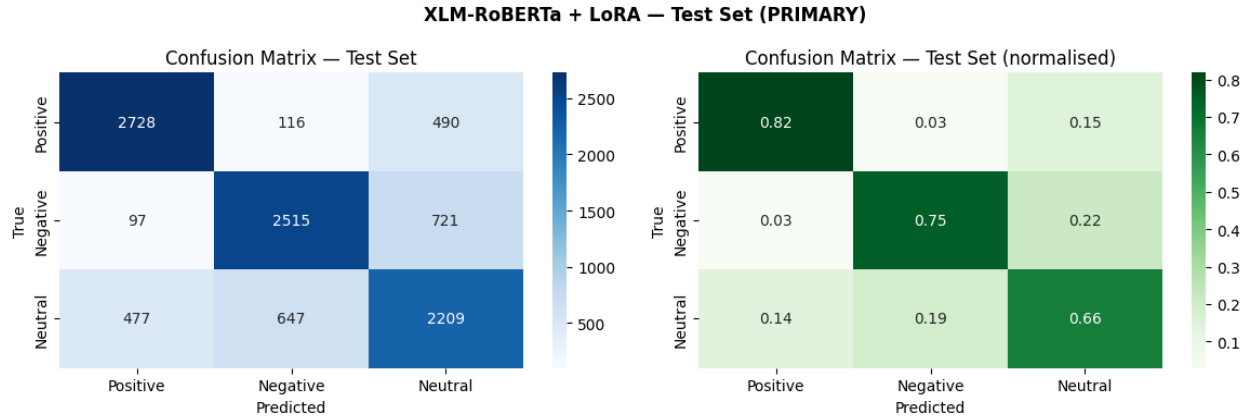
In this case also, the weighted F1 of the naturally written code-switched text dropped to 0.5936.
The pattern on the confusion matrix is the same in this case as well, with the model correctly identifying 98% of true Positive samples, 49% of true Negative and 19% of true Neutral, heavily defaulting to predicting Positive. This again proves that the sentiment cues are more subtle and context-dependent on naturally code-switched sentences compared to synthetic product reviews.

PEFT Results (Trainer API) using Classification Report

The LoRA model was evaluated on the same two test sets used for full fine-tuning: the synthetic test set (same domain as training) and the SentMix-3L natural test set (cross-domain generalisation benchmark).

Secondary Test Set (Synthetic)

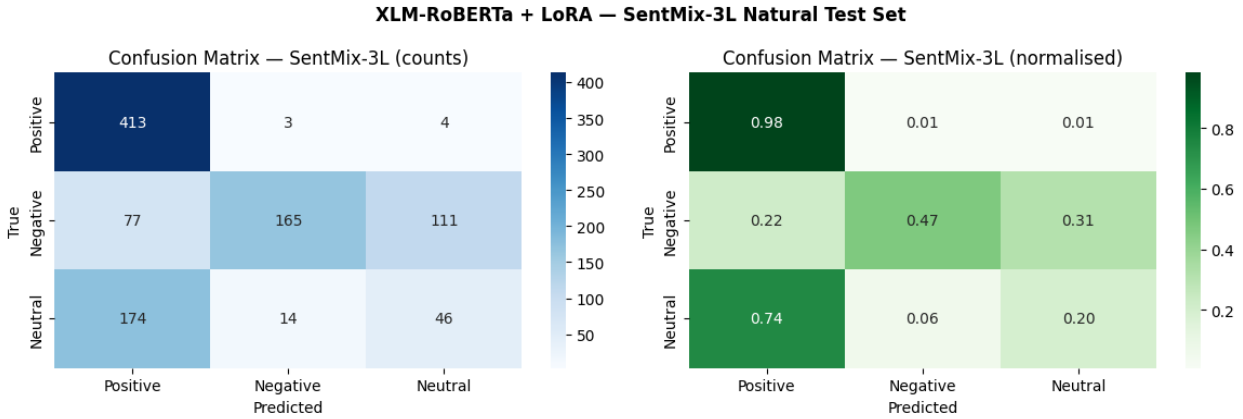
	precision	recall	f1-score	support
Positive	0.8262	0.8182	0.8222	3334
Negative	0.7672	0.7546	0.7609	3333
Neutral	0.6459	0.6628	0.6542	3333
accuracy			0.7452	10000
macro avg	0.7464	0.7452	0.7458	10000
weighted avg	0.7464	0.7452	0.7458	10000



The LoRA model achieved a weighted F1 of 0.7464 on the synthetic test set, which shares the same distribution as the training data. Performance was strongest on Positive (F1 = 0.822) and weakest on Neutral (F1 = 0.654).

Secondary Test Set (SentMix-3L Natural)

	precision	recall	f1-score	support
Positive	0.6220	0.9833	0.7620	420
Negative	0.9066	0.4674	0.6168	353
Neutral	0.2857	0.1966	0.2329	234
accuracy			0.6197	1007
macro avg	0.6048	0.5491	0.5372	1007
weighted avg	0.6436	0.6197	0.5882	1007



On the naturally written code-switched text, weighted F1 dropped to 0.5882. This gap of 15.9 points reflects the domain shift between algorithmically generated training data and authentic human code-switching. The confusion matrices reveal a similar pattern to full fine-tuning: the model correctly identified 98% of true Positive samples but only 47% of true Negative and 20% of true Neutral, heavily defaulting to predicting Positive.

Looking at the per-class breakdown, the model almost always predicts Positive - high recall but low precision. Negatives and Neutrals often suffer. Part of this is likely a sentiment bias in the synthetic training data. But another possible factor is the max length. We capped sequences at 192 tokens. If the sentiment-bearing part of a sentence appeared late in longer texts, it may have been truncated during training, meaning the model never actually saw the signal it needed to learn from.

PEFT Results (Native PyTorch) using Classification Report

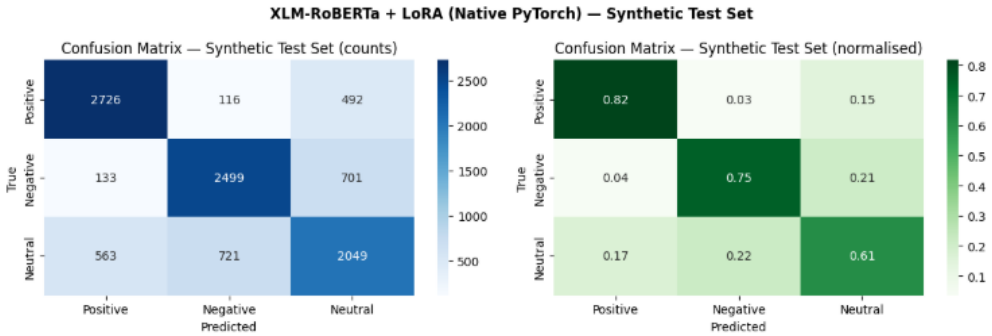
The same evaluation was performed on the LoRA model that was implemented from scratch using native PyTorch training loop.

Secondary Test Set (Synthetic)

```
── Synthetic Test Set ──────────────────────────
              precision    recall  f1-score   support

    Positive     0.7966    0.8176    0.8070      3334
    Negative     0.7491    0.7498    0.7494      3333
     Neutral     0.6320    0.6148    0.6233      3333

    accuracy                         0.7274     10000
   macro avg     0.7259    0.7274    0.7266     10000
weighted avg     0.7259    0.7274    0.7266     10000
```



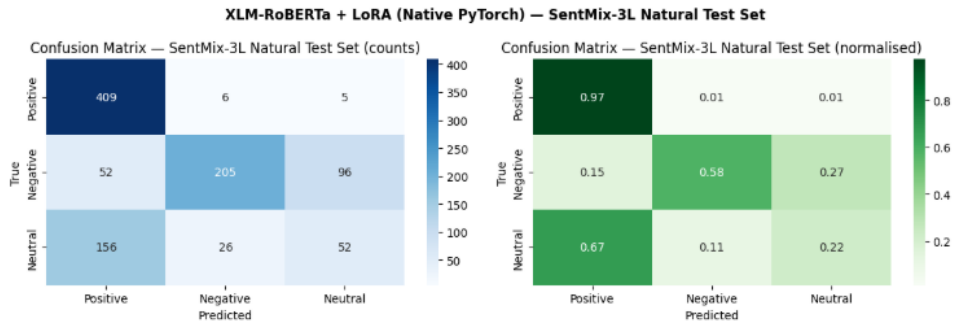
The weighted F1 in this case is 0.7266 which is again almost the same as the training data. Similar to the previous experiment, the performance was strongest on Positive (F1 = 0.8070) and weakest on Neutral (F1 = 0.6333).

Secondary Test Set (SentMix-3L Natural)

```
── SentMix-3L Natural Test Set ─────────────────────
              precision    recall  f1-score   support

    Positive     0.6629    0.9738    0.7888       420
    Negative     0.8650    0.5807    0.6949       353
     Neutral     0.3399    0.2222    0.2687       234

    accuracy                         0.6614      1007
   macro avg     0.6226    0.5923    0.5842      1007
weighted avg     0.6587    0.6614    0.6350      1007
```



In this case also, the weighted F1 of the naturally written code-switched text dropped to 0.6350.
The pattern on the confusion matrix is the same in this case as well, with the model correctly identifying 97% of true Positive samples, 58% of true Negative and 22% of true Neutral, heavily defaulting to predicting Positive. This again proves that the sentiment cues are more subtle and context-dependent on naturally code-switched sentences compared to synthetic product reviews.

Overall Evaluation: The Native PyTorch LoRA model achieved the highest weighted F1 of 0.6350 on the primary natural test set (SentMix-3L), outperforming both full fine-tuning variants (~0.5936) and the Trainer API LoRA (0.5882). This difference may be attributed to a few implementation choices. First, the shorter max sequence length of 102 tokens could have forced the model to focus on the most relevant parts of the sentence. Second, LoRA adapters were applied only to query and value projections rather than all 4 matrices in the Trainer API version, reducing overfitting.

Cultural Evaluation

Standard metrics like F1 and accuracy do not capture whether a model understands the cultural and linguistic nuances of code-switched text. We use naturally annotated test sets as culturally-aware benchmarks to assess whether errors are random or systematically tied to language-switching patterns.

For Hindi, the confusion matrices across all four implementations consistently show the model struggling with the Neutral class, frequently predicting Positive instead. This is not random — it reflects a cultural limitation specific to South Asian communication, where neutral or mixed sentiment is often conveyed through indirect language or understatement spread across two scripts. The model defaults to the dominant pattern in its synthetic training data rather than detecting these subtler cross-lingual cues. A further consideration is label quality: SentMix-3L labels were assigned by fluent human annotators, whereas the synthetic training labels were derived from Amazon star ratings — a proxy that does not fully capture how sentiment is expressed in authentic code-switched speech.

For Arabic, the EESA corpus consists of naturally written Egyptian Arabic-English YouTube comments with human-assigned labels, making it a culturally meaningful evaluation set. The model performed well overall, however, negative sentiment was consistently the weakest class across both Full Fine-Tuning and LoRA models, showing lower performance than the other classes. This may reflect the difficulty of detecting negative sentiment in dialectal Arabic, where criticism is often expressed indirectly or through culturally specific expressions that may not be well represented in XLM-RoBERTa's pretraining data.

Discussion

The two tasks produced different results on their respective test sets. Arabic achieved strong performance (~0.85 weighted F1) across all variants, while Hindi achieved comparable performance (~0.73 weighted F1) on the synthetic test set from the same Amazon distribution. As an additional evaluation, Hindi was also tested on SentMix-3L - a naturally written code-switched dataset from a completely different domain, where performance dropped to ~0.59-0.63 weighted F1, revealing the impact of domain shift when training on synthetic data and evaluating on authentic code-switched speech.

While the model performed reasonably across all three classes on the synthetic test set, this bias became severe on SentMix-3L, where the model defaulted to predicting Positive for the majority of samples. This suggests the synthetic Amazon reviews do not capture how sentiment is naturally expressed in real code-switched speech. A larger naturally annotated code-switched dataset for training would be the most

impactful improvement, though this lack of resources is precisely what creates the need for transfer learning in the first place.

Comparing frameworks, both Trainer API and Native PyTorch produced consistent results across both languages, confirming that the abstraction layer does not introduce meaningful performance differences. The exception is the Native PyTorch LoRA for Hindi, which achieved the best natural test F1 of 0.6350, likely due to its shorter max length and narrower LoRA target matrices reducing overfitting.

Hindi-Bangla-English

Approach	**Framework Style**	**Trainable Params**	**Trainable %**	**Training Time**	**Synthetic (Secondary) F1**	**Natural (Primary) F1**
Full Fine-Tuning	Trainer API	278M	100%	28.9 min	0.7356	0.5437
LoRA	Trainer API + PEFT	1.92M	0.69%	30.8 min	0.7458	0.5882
Full Fine-Tuning	Native PyTorch	278M	100%	22.8 min	0.7328	0.5936
LoRA	Native PyTorch from scratch	~887k	0.32%	72.3 min	0.7266	0.6350

Arabic-English

Approach	**Training Time (s)**	**Val F1**	**Test F1**
Full Fine-Tuning	618.49	0.8349	0.8341
LoRA	239.01	0.8260	0.8253

Conclusion and Future Work

This project demonstrated that transfer learning via XLM-RoBERTa can be effectively applied to sentiment analysis for low-resource code-switched languages under severe data constraints. For Arabic, strong results were achieved across all variants, with full fine-tuning reaching ~0.87 weighted F1 on the EESA corpus. For Hindi, performance was comparable on the synthetic test set (~0.73) but dropped significantly on the out-of-domain SentMix-3L benchmark, highlighting that synthetic training data alone is insufficient for robust generalisation to naturally code-switched text as the product reviews introduced domain mismatch and a positive sentiment bias.

Future work should focus on collecting larger naturally annotated code-switched datasets for both languages. For Hindi, exploring models pre-trained on South Asian languages such as MuRIL or IndicBERT may better capture trilingual code-switching patterns. For Arabic, extending evaluation to Gulf and Levantine dialects beyond Egyptian Arabic would improve generalisation across the broader Arabic-speaking world.

References

IBM 2026, *What is Fine Tuning?*, viewed 13 March 2026, https://www.ibm.com/think/topics/fine-tuning

TensorFlow.org 2026, *Transfer Learning and Fine Tuning*, viewed 13 March 2026, https://www.tensorflow.org/tutorials/images/transfer_learning

Raihan, Md Nishat and Goswami, Dhiman and Mahmud, Antara 2023, Mixed-Distil-BERT: Code-mixed Language Modeling for Bangla, English, and Hindi, viewed 12 March 2026, md-nishat-008/Code-Mixed-Sentiment-Analysis-Dataset · Datasets at Hugging Face

HuggingFace.co 2026, XLM-RoBERTa, viewed 13 March 2026, https://huggingface.co/transformers/v4.4.2/model_doc/xlmroberta.html

Raihan, Md Nishat and Goswami, Dhiman and Mahmud, Antara and Anstasopoulos, Antonios and Zampieri, Marcos 2023, SentMix-3L: A Bangla-English-Hindi Code-Mixed Dataset for Sentiment Analysis, viewed 20 March 2026, md-nishat-008/SentMix-3L · Datasets at Hugging Face

Md.Hamid Hosen, Yashfinul Haque, Dipannita Chakrobortty Nijhum, and Mohammed Nazim Uddin. 2025. XLM-RoBERTa-LIME: A Novel Explainable Framework for Sentiment Analysis on Bangla–English–Hindi Code-Mixed Text. In Proceedings of the 12th International Conference on Next Generation Computing, Communication, Systems and Security (NSysS '25). Association for Computing Machinery, New York, NY, USA, 44–52. https://doi.org/10.1145/3777555.3777560

Sherif, A., Sabty, C. (2025). Sentiment Analysis for Egyptian Arabic-English Code-Switched Data Using Traditional Neural Models and Advanced Language Models. In: Karpov, A., Delić, V. (eds) Speech and Computer. SPECOM 2024. Lecture Notes in Computer Science(), vol 15300. Springer, Cham. https://doi.org/10.1007/978-3-031-78014-1_5

GitHub Repository: ASTalaat 2026, EESA-Corpus, viewed 20 March 2026, https://github.com/ASTalaat/EESA-Corpus

Appendix

https://github.com/SaadiyahCodes/sentiment-analysis-316 - Docker + code

https://youtu.be/sHbGe7FsrD0 - Recorded Presentation

https://www.canva.com/design/DAHEgzX8Mwo/2plnt69uf6lXzYT-XLZY2w/edit?utm_content=DAHEgzX8Mwo&utm_campaign=designshare&utm_medium=link2&utm_source=sharebutton - Presentation